

CA03 Prototype – Presentation Q&A

MediConnect: Microservices Clinic Appointment Booking Platform (running prototype)

These are implementation/demo questions the panel is likely to ask in the 5-minute Q&A. Answer by pointing at the running system or the code, and be honest about simplifications.

A. The prototype & demo

Q1. What did you build, and how do you run it?

A running microservices prototype of MediConnect: a React single-page app, an API Gateway, and four Node.js/Express services (Auth, Appointment, Records, Notification), each with its own MongoDB database, plus RabbitMQ as an event bus. It is all containerised, so it starts with a single command, `docker compose up --build`, and the web app opens at `localhost:3000`.

Q2. Walk us through a booking in your running system.

The patient logs in and searches for a doctor; the request goes through the gateway, which verifies the JWT, to the Appointment Service. That service checks the slot is free, reserves it, records a simulated payment, saves the appointment, and publishes an `appointment.booked` event to RabbitMQ. The Notification Service consumes it and creates a confirmation, and the Records Service consumes the same event and creates a pending consultation record. The doctor later completes that record with notes and a prescription, which the patient can then view.

Q3. Which three major functionalities does the prototype demonstrate?

(1) User registration and login with JWT-based authentication and roles; (2) doctor search and appointment booking; and (3) consultation records with digital prescriptions. Notifications add a fourth flow that showcases the event-driven part.

B. Architecture in the code

Q4. How do your microservices actually communicate?

Two mechanisms. Synchronously over REST through the API Gateway for request/response, and there is one direct service-to-service REST call where the Records Service asks the Appointment Service to validate an appointment. Asynchronously through RabbitMQ: the Appointment Service publishes `appointment.booked`, and the Notification and Records services each consume it from their own queue – a fan-out.

Q5. Show us the event-driven part – how would you prove it works?

I open the RabbitMQ management UI at `localhost:15672`. After a booking, you can see the `mediconnect.events` exchange and two bound queues – one for notifications and one for records – each having received the message. In the container logs you also see the Appointment Service print “published `appointment.booked`” and the two consumers print that they received it.

Q6. How is each service structured internally?

Each service is layered: `routes.js` is the controller exposing the REST endpoints, `controller.js` holds the business logic and validation, `model.js` is the data-access layer using Mongoose, and `events.js` handles publishing or consuming RabbitMQ messages. This keeps a clear separation of concerns and matches the component diagram in our CA01 report.

Q7. You said database-per-service, but you only run one MongoDB container – explain.

Each service connects to its *own database* inside that MongoDB instance (`auth_db`, `appointment_db`, `records_db`, `notification_db`), and no service reads another's database. That gives the loose coupling of database-per-service. We kept a single MongoDB container to keep the prototype light; in production each could be a separate instance.

C. The API Gateway

Q8. What does the API Gateway do in your prototype?

It is the only public entry point. It verifies the JWT on every protected route, routes the request to the correct service, injects the authenticated identity as trusted headers (`x-user-id`, `x-user-role`), and applies rate limiting. The services themselves are not exposed on the host – they are only reachable inside the Docker network through the gateway.

Q9. How does the gateway know where to send each request?

It uses path-based routing with a reverse proxy: `/api/auth/*` goes to the Auth Service, `/api/doctors` and `/api/appointments` to the Appointment Service, `/api/records` to the Records Service, and `/api/notifications` to the Notification Service. It rewrites the path so, for example, `/api/doctors/123` arrives at the service as `/doctors/123`.

Q10. What was the hardest problem you solved?

The gateway path-rewriting. Because Express strips the mounted prefix, our first version sent the wrong path to the services (for example `/123` instead of `/doctors/123`), so the doctor search returned a 404 that the frontend showed as a parse error. We fixed it by rewriting the path with a function that re-adds the correct prefix, and verified it by checking the path each service actually receives.

D. Security, errors, deployment

Q11. How is security implemented in the prototype?

Passwords are hashed with `bcrypt`; login returns a signed JWT that carries the user id and role. The gateway verifies that token on every protected route and forwards the identity to services, which enforce role-based rules (only a patient can book, only a doctor can complete a record). The gateway also rate-limits requests. In production we would add TLS and encryption at rest, as described in CA01.

Q12. How do you handle errors?

Each service validates input and returns proper HTTP status codes (for example 400 for bad input, 401 for no token, 404 for not found, 409 for a taken slot), and every service and the gateway has a central error-handling middleware. The gateway also returns a clean 502 if a service is unreachable, and the

frontend shows friendly messages instead of crashing.

Q13. How is the prototype deployed, and how does it start reliably?

Docker Compose builds and runs seven containers – the four services, the gateway, the frontend, MongoDB and RabbitMQ. MongoDB and RabbitMQ have health checks, the services depend_on them being healthy, and each service also retries its database and broker connection, so the system comes up cleanly even though containers start at different times.

Q14. How does the frontend talk to the backend without CORS problems?

The React app is served by nginx, which proxies any /api request to the API Gateway. So the browser only ever calls its own origin (localhost:3000) and nginx forwards it internally – there is no cross-origin request, so no CORS configuration is needed.

E. Honesty & reflection

Q15. Which parts are simplified compared to your CA01 design?

The prototype is all Node.js (the design also mentioned Spring Boot), uses MongoDB only (the design also used PostgreSQL for transactional data), runs on Docker Compose rather than Kubernetes, simulates the payment step, and the doctor view shows all records rather than a single doctor's. These are deliberate scope reductions so we could demonstrate the architecture within the timeframe.

Q16. What happens if the Notification Service is down during the demo?

Booking still works. The appointment.booked event stays in its durable RabbitMQ queue, and when the Notification Service comes back it processes the backlog, so no confirmation is lost. That is exactly the availability benefit of the event-driven design.

Q17. How would you take this prototype toward production?

Deploy the same containers on Kubernetes with the Horizontal Pod Autoscaler and multiple gateway replicas, split the transactional data onto PostgreSQL, add Redis caching and TLS, integrate a real payment provider, and add centralised logging and tracing to follow a request across services.